```
// ColdChain Monitor - script.js
// Only uses topics from the Frontend Development Basics syllabus.

// ---------- 1. DATA (Module 3: Arrays & Objects) ----------
let shipments = [
  {
    id: "SHP-101", cargoType: "Fresh Produce", source: "Mumbai", destination: "Pune",
    minTemperature: 2, maxTemperature: 8,
    readings: [
      { timestamp: "01 Nov 08:00", temperature: 5.2 },
      { timestamp: "01 Nov 08:15", temperature: 6.1 },
      { timestamp: "01 Nov 08:30", temperature: 9.1 }
    ]
  },
  {
    id: "SHP-102", cargoType: "Frozen Seafood", source: "Chennai", destination: "Bengaluru",
    minTemperature: -22, maxTemperature: -18,
    readings: [
      { timestamp: "01 Nov 08:00", temperature: -20.1 },
      { timestamp: "01 Nov 08:15", temperature: -19.4 },
      { timestamp: "01 Nov 08:30", temperature: -15.1 }
    ]
  },
  {
    id: "SHP-103", cargoType: "Vaccines", source: "Delhi", destination: "Jaipur",
    minTemperature: 2, maxTemperature: 8,
    readings: [
      { timestamp: "01 Nov 08:00", temperature: 4.1 },
      { timestamp: "01 Nov 08:15", temperature: 4.5 },
      { timestamp: "01 Nov 08:30", temperature: 4.2 }
    ]
  },
  {
    id: "SHP-104", cargoType: "Dairy", source: "Ahmedabad", destination: "Surat",
    minTemperature: 1, maxTemperature: 5,
    readings: [
      { timestamp: "01 Nov 08:00", temperature: 3.2 },
      { timestamp: "01 Nov 08:15", temperature: 4.4 },
      { timestamp: "01 Nov 08:30", temperature: 6.9 }
    ]
  }
];

let alerts = [];

// ---------- 2. TEMPERATURE EVALUATOR (Module 3: Control Flow) ----------
// Returns "normal", "warning" or "critical"
function checkTemperature(temp, min, max) {
  if (temp >= min && temp <= max) {
    return "normal";
  }
  let difference;
  if (temp < min) {
    difference = min - temp;
  } else {
    difference = temp - max;
  }
  if (difference >= 3) {
    return "critical";
  }
  return "warning";
}

function getLatest(shipment) {
  return shipment.readings[shipment.readings.length - 1];
}

function findShipment(id) {
  for (let i = 0; i < shipments.length; i++) {
    if (shipments[i].id === id) {
```

```
      return shipments[i];
    }
  }
  return null;
}

// ---------- 3. ALERTS ----------
// Creates an alert for every out-of-range reading.
// The inner loop stops duplicate alerts for the same reading.
function createAlerts() {
  for (let i = 0; i < shipments.length; i++) {
    const s = shipments[i];
    for (let j = 0; j < s.readings.length; j++) {
      const r = s.readings[j];
      const status = checkTemperature(r.temperature, s.minTemperature, s.maxTemperature);
      if (status === "normal") {
        continue;
      }
      let exists = false;
      for (let k = 0; k < alerts.length; k++) {
        if (alerts[k].shipmentId === s.id && alerts[k].timestamp === r.timestamp) {
          exists = true;
        }
      }
      if (!exists) {
        alerts.push({
          id: "ALT-" + (alerts.length + 101),
          shipmentId: s.id,
          temperature: r.temperature,
          timestamp: r.timestamp,
          severity: status,
          status: "Active"
        });
      }
    }
  }
}

// ---------- 4. SAVE AND LOAD (localStorage + try/catch, Module 3: Error Handling) ----------
function saveData() {
  try {
    localStorage.setItem("coldchain", JSON.stringify({ shipments: shipments, alerts: alerts }));
  } catch (error) {
    console.log("Could not save data", error);
  }
}

function loadData() {
  try {
    const saved = localStorage.getItem("coldchain");
    if (saved) {
      const data = JSON.parse(saved);
      shipments = data.shipments;
      alerts = data.alerts;
      return true;
    }
  } catch (error) {
    showMessage("Shipment data could not be loaded.", "danger");
  }
  return false;
}

// ---------- 5. SMALL HELPER FUNCTIONS (Module 4: DOM Manipulation) ----------
function showMessage(text, type) {
  const box = document.getElementById("message");
  box.textContent = text;
  box.className = "alert alert-" + type;
}

// Show one screen and hide the others (classList)
```

```
function showSection(name) {
  const sections = ["dashboardSection", "detailsSection", "alertsSection"];
  for (let i = 0; i < sections.length; i++) {
    document.getElementById(sections[i]).classList.add("d-none");
  }
  document.getElementById(name).classList.remove("d-none");
}

// Status badge: text + icon, not only colour
function makeBadge(status) {
  const badge = document.createElement("span");
  if (status === "normal") {
    badge.className = "badge bg-success";
    badge.textContent = "✔ Normal";
  } else if (status === "warning") {
    badge.className = "badge bg-warning text-dark";
    badge.textContent = "⚠ Warning";
  } else {
    badge.className = "badge bg-danger";
    badge.textContent = "✖ Critical";
  }
  return badge;
}

function addCell(row, text) {
  const cell = document.createElement("td");
  cell.textContent = text;
  row.appendChild(cell);
}

function updateCounts() {
  let violations = 0;
  let unresolved = 0;
  for (let i = 0; i < shipments.length; i++) {
    const s = shipments[i];
    if (checkTemperature(getLatest(s).temperature, s.minTemperature, s.maxTemperature) !== "normal") {
      violations++;
    }
  }
  for (let i = 0; i < alerts.length; i++) {
    if (alerts[i].status !== "Resolved") {
      unresolved++;
    }
  }
  document.getElementById("totalShipments").textContent = shipments.length;
  document.getElementById("violationCount").textContent = violations;
  document.getElementById("unresolvedCount").textContent = unresolved;
  document.getElementById("alertCount").textContent = unresolved;
}

// ---------- 6. SCREEN 1: DASHBOARD ----------
function showDashboard() {
  const body = document.getElementById("shipmentBody");
  body.innerHTML = "";
  const searchText = document.getElementById("searchInput").value.toLowerCase();
  const filter = document.getElementById("statusFilter").value;

  for (let i = 0; i < shipments.length; i++) {
    const s = shipments[i];
    const latest = getLatest(s);
    const status = checkTemperature(latest.temperature, s.minTemperature, s.maxTemperature);

    if (filter !== "all" && filter !== status) {
      continue;
    }
    if (!(s.id + " " + s.cargoType).toLowerCase().includes(searchText)) {
      continue;
    }

    const row = document.createElement("tr");
```

```
    addCell(row, s.id);
    addCell(row, s.cargoType);
    addCell(row, s.source + " to " + s.destination);
    addCell(row, s.minTemperature + " to " + s.maxTemperature + " °C");
    addCell(row, latest.temperature + " °C");

    const statusCell = document.createElement("td");
    statusCell.appendChild(makeBadge(status));
    row.appendChild(statusCell);

    const buttonCell = document.createElement("td");
    const button = document.createElement("button");
    button.textContent = "View";
    button.className = "btn btn-sm btn-outline-primary view-btn";
    button.dataset.id = s.id;            // dataset: stores the shipment ID on the button
    buttonCell.appendChild(button);
    row.appendChild(buttonCell);

    body.appendChild(row);
  }
  updateCounts();
}

// ---------- 7. SCREEN 2: SHIPMENT DETAILS ----------
let selectedId = "";

function showDetails(id) {
  const s = findShipment(id);
  if (s === null) {
    showMessage("Temperature history could not be loaded.", "danger");
    return;
  }
  selectedId = id;
  const latest = getLatest(s);
  const status = checkTemperature(latest.temperature, s.minTemperature, s.maxTemperature);

  document.getElementById("detailTitle").textContent = s.id + " - " + s.cargoType;
  document.getElementById("detailInfo").textContent =
    s.source + " to " + s.destination + " | Safe range: " + s.minTemperature + " to " +
    s.maxTemperature + " °C | Latest: " + latest.temperature + " °C";

  const badgeBox = document.getElementById("detailBadge");
  badgeBox.innerHTML = "";
  badgeBox.appendChild(makeBadge(status));

  // History table: newest reading first
  const body = document.getElementById("historyBody");
  body.innerHTML = "";
  for (let i = s.readings.length - 1; i >= 0; i--) {
    const r = s.readings[i];
    const row = document.createElement("tr");
    addCell(row, r.timestamp);
    addCell(row, r.temperature + " °C");
    const cell = document.createElement("td");
    cell.appendChild(makeBadge(checkTemperature(r.temperature, s.minTemperature, s.maxTemperature)));
    row.appendChild(cell);
    body.appendChild(row);
  }
  showSection("detailsSection");
}

// ---------- 8. SCREEN 3: ALERTS ----------
function showAlerts() {
  const list = document.getElementById("alertList");
  list.innerHTML = "";
  const filter = document.getElementById("alertFilter").value;
  let shown = 0;

  for (let i = 0; i < alerts.length; i++) {
    const a = alerts[i];
```

```
    if (filter !== "all" && a.status !== filter) {
      continue;
    }
    shown++;

    const item = document.createElement("div");
    item.className = "list-group-item d-flex flex-wrap justify-content-between align-items-center gap-2";

    const info = document.createElement("span");
    info.textContent = a.shipmentId + " recorded " + a.temperature + " °C at " + a.timestamp;
    item.appendChild(info);

    const actions = document.createElement("div");
    actions.className = "d-flex flex-wrap align-items-center gap-2";
    actions.appendChild(makeBadge(a.severity));

    const statusTag = document.createElement("span");
    statusTag.className = "badge bg-secondary";
    statusTag.textContent = a.status;
    actions.appendChild(statusTag);

    if (a.status === "Active") {
      const ackBtn = document.createElement("button");
      ackBtn.textContent = "Acknowledge";
      ackBtn.className = "btn btn-sm btn-outline-primary";
      ackBtn.dataset.action = "ack";
      ackBtn.dataset.id = a.id;
      actions.appendChild(ackBtn);
    }
    if (a.status !== "Resolved") {
      const resBtn = document.createElement("button");
      resBtn.textContent = "Resolve";
      resBtn.className = "btn btn-sm btn-success";
      resBtn.dataset.action = "resolve";
      resBtn.dataset.id = a.id;
      actions.appendChild(resBtn);
    }

    item.appendChild(actions);
    list.appendChild(item);
  }

  if (shown === 0) {
    const empty = document.createElement("div");
    empty.className = "list-group-item text-muted";
    empty.textContent = "No temperature alerts. All monitored shipments are within their configured limit
      s.";
    list.appendChild(empty);
  }
}

function changeAlert(id, action) {
  for (let i = 0; i < alerts.length; i++) {
    if (alerts[i].id === id) {
      if (action === "ack") {
        alerts[i].status = "Acknowledged";
        showMessage("Alert Acknowledged", "success");
      } else {
        alerts[i].status = "Resolved";
        showMessage("Alert Resolved", "success");
      }
    }
  }
  saveData();
  showAlerts();
  updateCounts();
}

// ---------- 9. EVENTS (Module 4: Events & Event Handling) ----------
document.getElementById("navDashboard").addEventListener("click", function () {
```

```
  showDashboard();
  showSection("dashboardSection");
});

document.getElementById("navAlerts").addEventListener("click", function () {
  showAlerts();
  showSection("alertsSection");
});

document.getElementById("backBtn").addEventListener("click", function () {
  showDashboard();
  showSection("dashboardSection");
});

// Event delegation: one listener on the table body handles every "View" button
document.getElementById("shipmentBody").addEventListener("click", function (event) {
  if (event.target.classList.contains("view-btn")) {
    showDetails(event.target.dataset.id);
  }
});

// Event delegation for alert buttons
document.getElementById("alertList").addEventListener("click", function (event) {
  const action = event.target.dataset.action;
  if (action) {
    changeAlert(event.target.dataset.id, action);
  }
});

// Real-time search and filters
document.getElementById("searchInput").addEventListener("input", showDashboard);
document.getElementById("statusFilter").addEventListener("change", showDashboard);
document.getElementById("alertFilter").addEventListener("change", showAlerts);

// ---------- 10. FORM + VALIDATION (Module 4: Forms & Validation) ----------
document.getElementById("readingForm").addEventListener("submit", function (event) {
  event.preventDefault();                       // stop the page from reloading
  const input = document.getElementById("tempInput");
  const error = document.getElementById("tempError");
  const value = input.value.trim();

  if (value === "" || isNaN(Number(value))) {
    error.textContent = "Please enter a valid temperature number.";
    input.classList.add("is-invalid");
    return;
  }
  input.classList.remove("is-invalid");

  const s = findShipment(selectedId);
  s.readings.push({ timestamp: new Date().toLocaleString(), temperature: Number(value) });
  createAlerts();
  saveData();
  input.value = "";
  showDetails(selectedId);
  updateCounts();

  const status = checkTemperature(Number(value), s.minTemperature, s.maxTemperature);
  if (status === "normal") {
    showMessage("Temperature Within Range", "success");
  } else {
    showMessage("Temperature Threshold Exceeded", "warning");
  }
});

// Reset button
document.getElementById("resetBtn").addEventListener("click", function () {
  localStorage.removeItem("coldchain");
  location.reload();
});

```

```
// ---------- 11. START THE APP ----------
if (!loadData()) {
  createAlerts();
  saveData();
}
showDashboard();
showSection("dashboardSection");
```